

软件需求实验-第十三周

实验日期：5.28 姓名：胡江龙 学号：24302016

实验内容

作业1

首先，在 Reservation 类中增加了三种状态：RESERVED、CANCELLED 和 PAID，分别表示已预留、已取消和已支付。通过状态标记，可以在取消订单和确认支付时判断当前操作是否合法。

在创建库存预留时，程序会先检测预留数量是否合法，要求预留数量必须大于 0。同时，还会判断当前可售库存是否足够，如果可售库存小于申请预留数量，则直接抛出错误，不允许继续预留。只有当数据合法并且库存充足时，才会创建预留记录，然后减少可售库存，增加预留库存。这样可以保证预留后可售库存不会小于 0，从而避免超卖。

在取消订单时，程序会先检测预留记录的状态。如果该订单已经被取消，则不能重复取消；如果该订单已经完成支付，则不能再取消。只有当订单状态为 RESERVED 时，才允许取消。取消成功后，程序会将预留库存恢复到可售库存中，并把订单状态设置为 CANCELLED。

在确认支付时，程序同样会先检测订单状态。如果订单已经支付，则不能重复支付；如果订单已经取消，则不能再确认支付。只有当订单状态为 RESERVED 时，才允许确认支付。支付成功后，程序会减少预留库存和总库存，并将订单状态设置为 PAID。这样可以保证支付后的库存变化符合题目要求。

作业2

为了实现鲁棒性，我增强了 InventoryService 类，在其中增加了 _check_inventory()、_check_reservation() 和 _check_product_match() 三种检查方法。其中，_check_inventory() 用来检测库存数据是否合理，例如总库存、可售库存和预留库存不能小于 0，并且总库存应该等于可售库存与预留库存之和。_check_reservation() 用来检测预留记录是否合理，例如预留数量必须大于 0，预留状态必须是合法状态。_check_product_match() 用来检测预留记录和库存对象是否属于同一个商品，防止把一个商品的预留记录错误地用于另一个商品。

此外，我还在库存对象中增加了锁机制，使用 threading.RLock() 保证库存检查和库存修改能够作为一个整体执行。在创建预留、取消订单和确认支付时，都通过锁来保护库存数据，防止多个操作同时修改同一份库存数据，造成数据不一致或超卖问题。

在创建库存预留时，程序会先检测库存对象、用户 ID 和预留数量是否合法，再判断可售库存是否足够。如果数据不合理或库存不足，就会抛出错误。只有在数据合法时，才会创建预留记录并修改库存。

在取消订单时，程序会先检测库存数据、预留记录状态以及商品是否匹配。只有当数据合理并且订单处于 RESERVED 状态时，才会将预留库存恢复到可售库存，并将状态设置为 CANCELLED。

在确认支付时，程序也会先检测库存数据和订单状态。只有当订单处于 RESERVED 状态时，才会减少预留库存和总库存，并将状态设置为 PAID。

```
Inventory_reservation.py > Reservation > __init__
1  import uuid
2  import threading
3  class InventoryError(Exception):
4      """库存相关异常"""
5      pass
6  class Inventory:
7      def __init__(self, product_id, total_stock):
8          if not product_id:
9              raise InventoryError("product_id 不能为空")
10         if not isinstance(total_stock, int):
11             raise InventoryError("total_stock 必须是整数")
12         if total_stock < 0:
13             raise InventoryError("total_stock 不能小于 0")
14         self.product_id = product_id
15         self.total_stock = total_stock
16         self.available_stock = total_stock
17         self.reserved_stock = 0
18         # 用锁保证库存检查和库存修改是一个整体操作, 避免并发情况下超卖
19         self._lock = threading.RLock()
20     def __repr__(self):
21         return (
22             f"Inventory(product_id={self.product_id}, "
23             f"total_stock={self.total_stock}, "
24             f"available_stock={self.available_stock}, "
25             f"reserved_stock={self.reserved_stock})"
26         )
27 class Reservation:
28     STATUS_RESERVED = "RESERVED"
29     STATUS_CANCELLED = "CANCELLED"
30     STATUS_PAID = "PAID"
31     def __init__(self, product_id, user_id, quantity):
32         if not product_id:
33             raise InventoryError("product_id 不能为空")
34         if not user_id:
35             raise InventoryError("user_id 不能为空")
36         if not isinstance(quantity, int):
37             raise InventoryError("quantity 必须是整数")
38         if quantity <= 0:
39             raise InventoryError("quantity 必须大于 0")
40         self.id = str(uuid.uuid4())
41         self.product_id = product_id
42         self.user_id = user_id
43         self.quantity = quantity
44         self.status = Reservation.STATUS_RESERVED
45     def __repr__(self):
46         return (
47             f"Reservation(id={self.id[:8]}..., "
48             f"product_id={self.product_id}, "
49             f"user_id={self.user_id}, "
50             f"quantity={self.quantity}, "
51             f"status={self.status})"
52         )
```

```
inventory_reservation.py > Inventory > __repr__
27 class Reservation:
45     def __repr__(self):
52 )
53 class InventoryService:
54     def _check_inventory(self, inventory):
55         if inventory is None:
56             raise InventoryError("inventory 不能为空")
57         if inventory.total_stock < 0:
58             raise InventoryError("total_stock 不能小于 0")
59         if inventory.available_stock < 0:
60             raise InventoryError("available_stock 不能小于 0")
61         if inventory.reserved_stock < 0:
62             raise InventoryError("reserved_stock 不能小于 0")
63         if inventory.total_stock != inventory.available_stock + inventory.reserved_stock:
64             raise InventoryError("库存数据不一致")
65     def _check_reservation(self, reservation):
66         if reservation is None:
67             raise InventoryError("reservation 不能为空")
68         if reservation.quantity <= 0:
69             raise InventoryError("reservation.quantity 必须大于 0")
70         if reservation.status not in {
71             Reservation.STATUS_RESERVED,
72             Reservation.STATUS_CANCELLED,
73             Reservation.STATUS_PAID,
74         }:
75             raise InventoryError("reservation.status 状态非法")
76     def _check_product_match(self, inventory, reservation):
77         if inventory.product_id != reservation.product_id:
78             raise InventoryError("预订记录与库存商品不匹配")
79     def reserve(self, inventory, user_id, quantity):
80         if inventory is None:
81             raise InventoryError("inventory 不能为空")
82         if not user_id:
83             raise InventoryError("user_id 不能为空")
84         if not isinstance(quantity, int):
85             raise InventoryError("quantity 必须是整数")
86         if quantity <= 0:
87             raise InventoryError("预留数量必须大于 0")
88         with inventory._lock:
89             self._check_inventory(inventory)
90             if inventory.available_stock < quantity:
91                 raise InventoryError("可售库存不足, 不能创建预留")
92             reservation = Reservation(inventory.product_id, user_id, quantity)
93             inventory.available_stock -= quantity
94             inventory.reserved_stock += quantity
95             self._check_inventory(inventory)
96             return reservation
97     def cancel(self, inventory, reservation):
98         self._check_inventory(inventory)
99         self._check_reservation(reservation)
100         self._check_product_match(inventory, reservation)
101         with inventory._lock:
102             self._check_inventory(inventory)
103             if reservation.status == Reservation.STATUS_CANCELLED:
104                 raise InventoryError("已取消的预留不能重复取消")
105             if reservation.status == Reservation.STATUS_PAID:
106                 raise InventoryError("已完成支付的预留不能取消")
107             if reservation.status != Reservation.STATUS_RESERVED:
108                 raise InventoryError("只有 RESERVED 状态的预留才能取消")
109             if inventory.reserved_stock < reservation.quantity:
110                 raise InventoryError("预留库存不足, 库存数据异常")
111             inventory.available_stock += reservation.quantity
112             inventory.reserved_stock -= reservation.quantity
113             reservation.status = Reservation.STATUS_CANCELLED
114             self._check_inventory(inventory)
115     def confirm_payment(self, inventory, reservation):
116         self._check_inventory(inventory)
117         self._check_reservation(reservation)
118         self._check_product_match(inventory, reservation)
```

软

实验

实

代码

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52

```

118         self._check_product_match(inventory, reservation)
119     with inventory._lock:
120         self._check_inventory(inventory)
121         if reservation.status == Reservation.STATUS_PAID:
122             self._check_inventory(inventory)
123     def confirm_payment(self, inventory, reservation):
124         self._check_inventory(inventory)
125         self._check_reservation(reservation)
126         self._check_product_match(inventory, reservation)
127     with inventory._lock:
128         self._check_inventory(inventory)
129         if reservation.status == Reservation.STATUS_PAID:
130             raise InventoryError("已支付的预留不能重复支付")
131         if reservation.status == Reservation.STATUS_CANCELLED:
132             raise InventoryError("已取消的预留不能确认支付")
133         if reservation.status != Reservation.STATUS_RESERVED:
134             raise InventoryError("只有 RESERVED 状态的预留才能确认支付")
135         if inventory.reserved_stock < reservation.quantity:
136             raise InventoryError("预留库存不足, 库存数据异常")
137         inventory.reserved_stock -= reservation.quantity
138         inventory.total_stock -= reservation.quantity
139         reservation.status = Reservation.STATUS_PAID
140         self._check_inventory(inventory)
141 if __name__ == "__main__":
142     service = InventoryService()
143     inventory = Inventory("product_001", 10)
144     print("初始库存: ")
145     print(inventory)
146     reservation1 = service.reserve(inventory, "user_001", 3)
147     reservation2 = service.reserve(inventory, "user_002", 4)
148     print("\n创建预留后:")
149     print(inventory)
150     print(reservation1)
151     print(reservation2)
152     service.cancel(inventory, reservation1)
153     print("\n取消 reservation1 后: ")
154     print(inventory)
155     print(reservation1)
156     service.confirm_payment(inventory, reservation2)
157     print("\n确认支付 reservation2 后: ")
158     print(inventory)
159     print(reservation2)

```

作业3

我将不同优惠券封装成不同的类，FullReductionCoupon 表示满减券，DiscountCoupon 表示折扣券，NewUserCoupon 表示新用户券，CategoryCoupon 表示品类券，VipCoupon 表示 VIP 券，ShippingCoupon 表示运费券。每个优惠券类都有自己的 calculate() 方法，用来计算对应的优惠金额。

CouponService 不再关心具体是哪种优惠券，只负责遍历优惠券对象列表，并调用每个优惠券对象的 calculate(user, order) 方法。这样不同优惠券可以自由组合，例如满减券、新用户券、VIP券和运费券可以一起使用。

如果以后新增一种优惠券，只需要新增一个优惠券类，并实现 calculate() 方法即可，不需要修改 CouponService 的核心代码。因此，这种设计提高了程序的灵活性和重用性。

coupon_service.py > CouponService > calculate_discount

```
1 class User:
2     def __init__(self, is_new=False, is_vip=False):
3         self.is_new = is_new
4         self.is_vip = is_vip
5 class Order:
6     def __init__(self, amount, category=None, shipping_fee=0):
7         self.amount = amount
8         self.category = category
9         self.shipping_fee = shipping_fee
10 class FullReductionCoupon:
11     """满减券: 满300减50"""
12     def calculate(self, user, order):
13         if order.amount >= 300:
14             return 50
15         return 0
16 class DiscountCoupon:
17     """折扣券: 8折, 相当于优惠20%"""
18     def calculate(self, user, order):
19         return order.amount * 0.2
20 class NewUserCoupon:
21     """新用户券: 新用户减20"""
22     def calculate(self, user, order):
23         if user.is_new:
24             return 20
25         return 0
26 class CategoryCoupon:
27     """品类券: 生鲜满100减10"""
28     def calculate(self, user, order):
29         if order.category == "生鲜" and order.amount >= 100:
30             return 10
31         return 0
32 class VipCoupon:
33     """VIP券: VIP 95折, 相当于优惠5%"""
34     def calculate(self, user, order):
35         if user.is_vip:
36             return order.amount * 0.05
37         return 0
38 class FlashSaleCoupon:
39     """秒杀券: 限时优惠30"""
40     def calculate(self, user, order):
41         return 30
42 class ShippingCoupon:
43     """运费券: 包邮"""
44     def calculate(self, user, order):
45         return order.shipping_fee
```

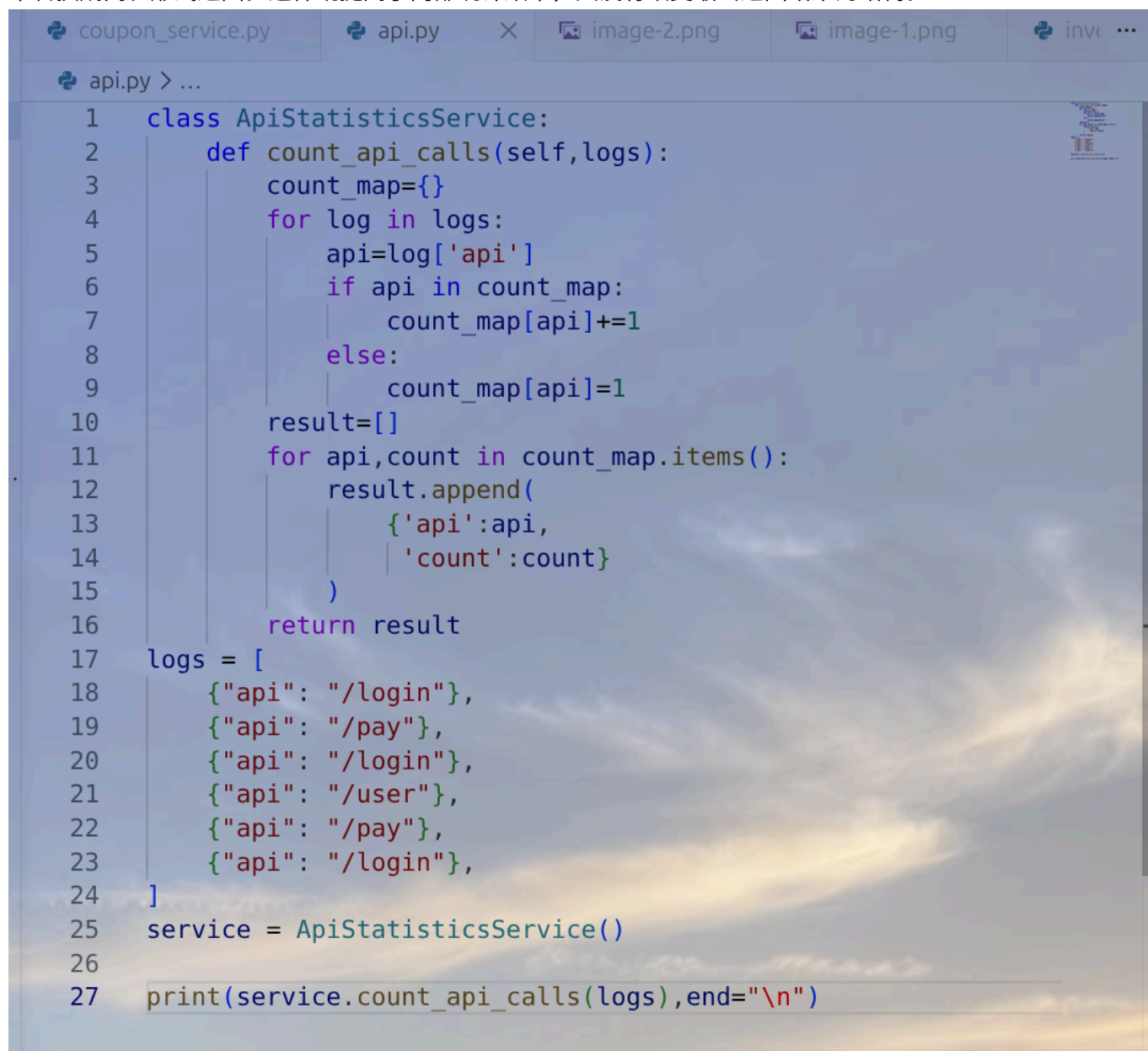
```
39     """秒杀券：限时优惠30"""
40     def calculate(self, user, order):
41         return 30
42 class ShippingCoupon:
43     """运费券：包邮"""
44     def calculate(self, user, order):
45         return order.shipping_fee
46 class CouponService:
47     def calculate_discount(self, user, order, coupons):
48         total_discount = 0
49         for coupon in coupons:
50             total_discount += coupon.calculate(user, order)
51         max_discount = order.amount + order.shipping_fee
52         if total_discount > max_discount:
53             return max_discount
54         return total_discount
55 if __name__ == "__main__":
56     user = User(is_new=True, is_vip=True)
57     order = Order(amount=350, category="生鲜", shipping_fee=12)
58     service = CouponService()
59     coupons = [
60         FullReductionCoupon(),
61         NewUserCoupon(),
62         CategoryCoupon(),
63         VipCoupon(),
64         ShippingCoupon()
65     ]
66     discount = service.calculate_discount(user, order, coupons)
67     print("订单金额: ", order.amount)
68     print("运费: ", order.shipping_fee)
69     print("优惠金额: ", discount)
70     print("最终支付: ", order.amount + order.shipping_fee - discount)
```

作业4

Todo4 高效性修改说明

我将原来的列表查找方式改成了字典统计方式。字典 count_map 使用 API 地址作为 key，访问次数作为 value。每读取一条日志，就直接通过 API 名称在字典中查找和更新访问次数。如果该 API 已经存在，就将次数加 1；如果不存在，就把它加入字典，并将次数设置为 1。修改后，程序不需要每次都遍历整个统计结果列表，而是可以通过字典快速定位某个 API 的统计结果。因此，统计过程只需要遍历一遍日志数据即可完成。这样可以明显减少重复查找，提高日志统计效率。最后，为了保持输出格式和原程序一致，再将字典中的统计结

果转换成列表形式返回。这样既提高了内部统计效率，又没有改变最终返回结果的结构。

A screenshot of a code editor with a dark theme. The editor has several tabs at the top: 'coupon_service.py', 'api.py' (which is the active tab), 'image-2.png', 'image-1.png', and 'inv...'. The code in the 'api.py' tab is as follows:

```
1 class ApiStatisticsService:
2     def count_api_calls(self, logs):
3         count_map={}
4         for log in logs:
5             api=log['api']
6             if api in count_map:
7                 count_map[api]+=1
8             else:
9                 count_map[api]=1
10        result=[]
11        for api,count in count_map.items():
12            result.append(
13                {'api':api,
14                 'count':count}
15            )
16        return result
17    logs = [
18        {"api": "/login"},
19        {"api": "/pay"},
20        {"api": "/login"},
21        {"api": "/user"},
22        {"api": "/pay"},
23        {"api": "/login"},
24    ]
25    service = ApiStatisticsService()
26
27    print(service.count_api_calls(logs),end="\n")
```